



25-PACKAGING-UPDATE-AND-ROLLBACK.md — Shipping a fix, and undoing a bad one



O3 — Every version can be undone. A desktop app has no "revert the deploy" button. Once a bad build is on 400 machines, the only thing that saves you is a rollback path you designed *before* you needed it.

1. What ships

Platform	Artefact	Signing	Size target
Windows 10/11 x64	MSI + portable ZIP	Authenticode, EV certificate	≤ 180 MB installer
macOS 12+ (arm64 + x64)	Universal DMG	Developer ID + notarised + stapled	≤ 220 MB
Linux x64	AppImage + .deb	GPG detached signature	≤ 180 MB

Models are not in the installer. Roughly 2–4 GB of ONNX weights ship as a separate, resumable, checksummed download on first run. Bundling them makes the installer hostile on a Nepali or Indonesian connection and forces a full reinstall for a model-only fix. Model packs are versioned independently and cached in a shared directory so a rollback does not re-download 3 GB.

An EV code-signing certificate costs roughly USD 300–500 per year and is not optional on Windows — without it, SmartScreen shows "unknown publisher" and your conversion rate collapses. macOS notarisation is free but adds 5–20 minutes to release. Both must run on a clean VM in CI, never from a laptop.

2. Version scheme

`MAJOR.MINOR.PATCH+build.<git-sha>`, plus three independently versioned things that users will conflate and you must not:

- **App version** — the binary.
- **Catalog schema version** — the SQLite migration number. Only ever increases.
- **Model pack version** — per model family, from the model cards.

The compatibility rule that makes rollback possible:

App version N must be able to open a catalog written by app version N+1, for at least one minor version, by ignoring fields it does not understand. A migration that rewrites data destructively is forbidden without a `PRE-MIGRATION BACKUP`.

Without this rule, rollback means data loss, which means rollback is not real.

3. The release channels

Channel	Audience	Cadence	Gate
<code>nightly</code>	You only	Every green main	Full nightly CI lane
<code>beta</code>	25–50 partner photographers	Weekly	Nightly lane + one real wedding processed end to end
<code>stable</code>	Everyone	Every 2–4 weeks	Two weeks on beta with zero RunBlocking errors
<code>lts</code>	Studios mid-season	Quarterly	Security fixes only

Never ship on a Friday, and never ship in peak wedding season without an LTS available. A photographer with 40 weddings booked between May and October cannot absorb a broken update; the `lts` channel is how you keep their trust.

4. Staged rollout — frozen contract

```
// crates/aura-update/src/contract/rollout.rs -- FROZEN CONTR
ACT
pub struct RolloutPlan {
```

```

pub version: SemVer,
pub channel: Channel,
/// Percentage cohorts, advanced only when health holds.
pub stages: Vec<RolloutStage>, // e.g. 1%, 5%, 25%, 10
0%
pub health_gate: HealthGate,
}

pub struct HealthGate {
    /// Halt if crash rate exceeds baseline by this factor.
    pub max_crash_ratio: f32, // 1.5
    /// Halt on any new Fatal or RunBlocking code not seen in
the prior version.
    pub halt_on_new_blocking_code: bool, // true
    /// Minimum sessions observed before advancing a stage.
    pub min_sessions: u32, // 200
    pub min_hours: u32, // 24
}

pub enum RolloutState { Pending, Active, Halted { reason: String }, Complete, RolledBack }

```

Cohort assignment is `blake3(install_uuid || version) % 100` — deterministic, so a machine stays in the same cohort across checks, and no server-side user list is needed. This matters: staged rollout must work without you holding a database of who your users are.

5. The update flow, from the user's side

1. A background check, at most once every 6 hours, over HTTPS with certificate pinning, fetching a **signed manifest** — never an executable.
2. The manifest is verified against a public key compiled into the binary. An unsigned or mis-signed manifest is `AURA-SEC-9004` and is discarded silently.
3. Download is resumable, checksummed, and happens in the background at low priority. **Never during an active run** — competing for disk and CPU while

someone processes 4,000 RAWs is unforgivable.

4. The user is asked. There is no silent auto-update in a professional tool; an update that changes edit output mid-project is a broken promise about determinism.
5. Install happens on next launch, with the previous version preserved on disk.
6. **First launch after update runs a self-test:** open the catalog, decode one RAW, run one model, render one preview, verify a golden digest. If the self-test fails, offer automatic rollback immediately rather than letting the user discover the breakage on a real wedding.

6. Rollback — the part everyone skips

The five-minute promise: a non-technical user must be able to return to the previous version in under five minutes without losing work.

- **Keep N-1 installed.** Disk is cheap; a broken season is not. The previous version stays in `versions/<semver>/` and is launchable from a Help > Revert menu item.
- **Back up the catalog before every migration,** to `backups/pre-<schema-version>-<timestamp>.sqlite`, verified with `PRAGMA integrity_check` before the migration proceeds. Retain the last 5.
- **Down-migrations for one schema version.** Beyond that, restore from the pre-migration backup — which is why the backup is mandatory rather than best-effort.
- **Recipes are forward-compatible JSON** with a version field and unknown-field preservation, so an older app opening a newer recipe keeps the fields it cannot render rather than dropping them.
- **Server-side kill switch:** the signed manifest can mark a version `revoked`, which makes clients on that version show a banner with a one-click revert. This is your only lever once a bad build is out; test it on a real user before you need it.
- **Model rollback is independent.** A bad model pack reverts without touching the app, and the previous pack is retained on disk.

7. The release checklist

- ☐ All CI gates green on the nightly lane, including determinism and `model-gate`.
- ☐ Installer built on a clean VM, signed, and — on macOS — notarised and stapled.
- ☐ Installed on a **fresh** machine per platform, no dev tools present.
- ☐ Upgrade tested from the two previous versions with a real 4,000-image catalog.
- ☐ **Rollback tested**, $N \rightarrow N-1$, with the catalog opening intact afterwards.
- ☐ Self-test passes on all three platforms.
- ☐ Migration tested on a 200,000-row catalog; duration measured and shown to the user if over 5 seconds.
- ☐ Antivirus false-positive check (Windows Defender, and a VirusTotal scan — Rust binaries get flagged more than you would expect).
- ☐ Release notes written for photographers, not for engineers: what changed in *their* results.
- ☐ Any change to edit output flagged prominently, because it breaks determinism expectations for in-flight projects.
- ☐ Rollout plan configured with cohorts and health gate.
- ☐ Previous version's download URL still live.

8. Tests

- ☐ **Signature tamper test:** a manifest with one flipped byte is rejected.
- ☐ **Downgrade test:** $N-1$ opens a catalog written by N and does not corrupt it; verified by digest before and after.
- ☐ **Interrupted update test:** kill the process during download, during extract, and during migration. All three resume or roll back cleanly, never leaving a half-installed app.
- ☐ **Disk-full during update** leaves the old version working.

- ☐ **Offline test:** no network for 30 days; the app works normally and never nags.
- ☐ **Kill-switch test:** mark a version revoked; assert clients show the banner and revert successfully.
- ☐ **Migration crash test:** kill during migration; assert the pre-migration backup restores automatically on next launch.

9. Acceptance criteria

- ☐ `rollout.rs` matches section 4 and is under contract freeze.
- ☐ Installers are reproducible: two CI runs of the same commit produce identical hashes, excluding the signature block.
- ☐ N-1 remains installed and launchable after every update.
- ☐ Every migration writes a verified backup first.
- ☐ The self-test runs on first launch after update and can trigger rollback.
- ☐ The revocation kill switch has been exercised end to end at least once on the beta channel.
- ☐ Rollback from stable to the previous stable has been performed by someone who did not write the code, in under five minutes, timed.

10. Claude Code prompt

Act as `DEVOPS` (Release Engineering) with `TLC` reviewing. Implement `25-PACKAGING-UPDATE-AND-ROLLBACK.md`: the Tauri bundle configuration for all three platforms, the signed-manifest update client with certificate pinning and deterministic cohort assignment, the separate resumable model-pack downloader, the pre-migration backup with integrity verification, the post-update self-test, and the Help > Revert flow that launches N-1.

Constraints: never download or install during an active processing run; never auto-install without consent; the manifest public key is compiled in, not fetched; every filesystem operation during install must be atomic or recoverable.

Then run the interrupted-update test at all three kill points and paste the outcomes, and demonstrate a full $N \rightarrow N-1$ rollback with a 4,000-image catalog opening intact afterwards, with the digest before and after.